

1. 答: 记分牌算法基本思想: 通过记分牌硬件实现对指令的动态调度。该硬件中维护着三张表, 分别用于记录指令的执行状态、功能部件状态、寄存器状态以及数据相关关系等。把一段流水线中的译码段 ID 分解成了两个段: 流出和读指令数, 以避免当某条指令在 ID 段被停顿时堵住后面无冲突指令的流动。记分牌的目标是在没有结构冲突时, 尽可能早地执行没有数据冲突的指令, 实现每个时钟周期执行一条指令。如果某条指令被暂停, 而后面的指令与流水线中正在执行或被暂停的指令不相关, 那么这些指令可以跨越它们, 继续流出或执行下去, 故尽管所有的指令在流水段是按序流出的, 但可能是乱序执行和乱序完成的。采用集中式硬件机制, 通过互锁机制消除相关, 指令执行时满足没有冲突和操作指令数就绪的条件。

2. 答: Tomasulo 算法基本思想: 目标是在没有专用编译器的情况下, 提高系统性能。核心思想是①记录和检测指令相关, 操作数一旦就绪就立即执行, 把发生 RAW 冲突的可能性减少到最小; ②通过寄存器栈名来消除 WAR 冲突和 WAW 冲突。具体思想: 只要操作数有效, 就将其取到保留站, 避免指令流出时才到寄存器中取数据。这就使得即将执行的指令从相应的保留站中取得操作数, 而不是从寄存器中。指令的执行结果也是直接送到等待其数据的其它保留站中去。因而, 对于连续的寄存器写, 只有最后一个才真正更新寄存器中的内容。一条指令流出时, 存放操作数的寄存器名被换成对应于该寄存器保留站的名称。

3. 若记号解算法需以个时钟周期

	issue	operands	complete	result
L.D F6, 34(R2)	1	2	3	4
L.D F2, 44(R2)	2	3	4	5
ADD.D F8, F6, F2	3	6	8	9
MUL.D F4, F2, F6	4	6	16	17
SUB.D F8, F4, F2	10	18	20	21
ADD.D F4, F2, F6	18	19	21	22

第5个时钟周期

	issue	operands	complete	result
L.D F6, 34(R2)	1	2	3	4
L.D F2, 44(R2)	2	3	4	5
ADD.D F8, F6, F2	3			
MUL.D F4, F2, F6	4			
SUB.D F8, F4, F2				
ADD.D F4, F2, F6				

	busy	op	F _i	F _j	F _k	R _j	R _k	R _j	R _k
integer1	N								
integer2	N								
add 1	Y	add	F8	F6	F2			Yes	Yes
add 2	N								
mult 1	Y	mult	F4	F2	F6			Yes	Yes
mult 2	N								
clock			F0	F2	F4	F6	F8	F10	--
J					mult1		add1		

第10个时钟周期

	issue	operands	complete	result
L.D F6, 34(Rv)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
ADD.D F8, F6, F2	3	6	8	9
MUL.D F4, F2, F6	4	6		
SUB.D F8, F4, F2	10			
ADD.D F4, F2, F6				

	busy	op	F _i	F _j	F _k	(R _j)	(R _k)	R _j	R _k
integer1	N								
integer2	N								
add1	Y	sub	F8	F4	F2	mult		No	Yes
add2	N								
mult1	Y	mult	F4	F2	F6			No	No
mult2	N								
clock	T0	F2	F4	F6	F8	F10	...		
10		mult1				add1			

第15个时钟周期

	issue	operands	complete	result
L.D F6, 34(Rv)	1	2	3	4
L.D F2, 45(R3)	2	3	4	5
ADD.D F8, F6, F2	3	6	8	9
MUL.D F4, F2, F6	4	6		
SUB.D F8, F4, F2	10			
ADD.D F4, F2, F6				

	busy	op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
integer 1	N								
integer 2	N								
add 1	Y	sub	F ₈	F ₄	F ₂	mult		No	Yes
add 2	N								
mult 1	Y	mult	F ₄	F ₂	F ₆			No	No
mult 2	N								
clock	T0	F ₂	F ₄	F ₆	F ₈	F ₁₀	--		
15			mult 1		add 1				

4. 答: Tomasulo 算法需 18 个时钟周期

	issue	Compl	Result
L.D F ₆ , 34 (R ₂)	1	2	3
L.D F ₂ , 45 (R ₃)	2	3	4
ADD.D F ₈ , F ₆ , F ₂	3	6	7
MUL.D F ₄ , F ₂ , F ₆	4	14	5
SUB.D F ₈ , F ₄ , F ₂	5	17	18
ADD.D F ₄ , F ₂ , F ₆	8	10	11

第 5 个时钟周期

	issue	Compl	result
L.D F ₆ , 34 (R ₂)	1	2	3
L.D F ₂ , 45 (R ₃)	2	3	4
ADD.D F ₈ , F ₆ , F ₂	3		
MUL.D F ₄ , F ₂ , F ₆	4		

SUB.D F8, F4, F2 J

ADD.D F4, F2, F6

busy address

load1 No

load2 No

busy op Vj V_k Qj Q_k

add1 Yes add Mem[34+R₂] Mem[45+R₂]

add2 Yes sub Regs[F₄] Mem[45+R₂]

mult1 Yes mult Mem[45+R₂] Mem[34+R₂]

mult2 No

clock F0 F2 F4 F6 F8 F10 ---

T mult1 add2

第10个时钟周期

issue

compl

result

L.D F6, 34(R₂)

1

2

3

L.D F2, 45(R₂)

2

3

4

ADD.D F8, F6, F2

3

6

7

MUL.D F4, F2, F6

4

SUB.D F8, F4, F2

J

ADD.D F4, F2, F6

8

10

busy address

load1 No

load2 No

	busy	op	V_j	V_k	$(2)_j$	$(2)_k$
add1	Yes	add	$\text{Mem}[45+R_3]$	$\text{Mem}[34+R_2]$		
add2	Yes	sub	$\text{Regs}[R_4]$	$\text{Mem}[45+R_3]$		
mult1	Yes	mult	$\text{Mem}[45+R_3]$	$\text{Mem}[34+R_2]$		
mult2	No					

clock	F0	F2	F4	F6	F8	F10	...
10			add1		add2		

第15个时钟周期

	issue	compl	result
L.D F6, 34(R2)	1	2	3
L.D F2, 45(R3)	2	3	4
ADD.D F8, F6, F2	3	6	7
MUL.D F4, F2, F6	4	14	15
SUB.D F8, F4, F2	5		
ADD.D F4, F2, F6	8	10	11

	busy	address
load1	No	
load2	No	

	busy	op	V_j	V_k	$(2)_j$	$(2)_k$
add1	No					
add2	Yes	sub	$\text{Regs}[F_4]$	$\text{Mem}[45+R_3]$		
mult1	No					
mult2	No					

clock F0 F2 F4 F6 F8 ...
15 addr

答: Tomasulo 算法需要时钟周期更少

① 通过保留站和 CBR 实现分布的冲突检测逻辑。在 Tomasulo 算法中, 如果有多条指令已经获得了一个操作数, 并同时在等待同一运算结果, 那么这个结果一旦产生, 就可以通过 CBR 同时广播给所有这些指令, 使它们可以同时执行。而记号牌算法采用的是集中的寄存器组, 各条指令就要等结果写入寄存器, 然后再依次顺序从寄存器组读出。

② 通过使用保留站进行寄存器改名, 并且在操作数一旦就绪就将之写入保留站中实现, 消除了 WAW 冲突和 WAR 冲突导致的停顿。